



Частное профессиональное образовательное учреждение  
«КОЛЛЕДЖ СОВРЕМЕННЫХ ТЕХНОЛОГИЙ И МЕДИЦИНЫ»

## Междисциплинарная аттестационная работа.

на тему «ИС «Кинотеатр» (Залы, Фильмы, Сеансы, Билеты).»

формы обучения **очная**

подчеркнуть нужное

специальность 09.02.07 Информационные системы и программирование  
(код и наименование)

Обучающийся  
группы

Е.Э.Афоница  
К.О.Козина  
Э.И.Штейн

подпись, дата

И.О. Фамилия

инициалы и фамилия

Оценка выполненной обучающимся работы: \_\_\_\_\_

Преподаватель,  
уч. степ., уч. зв.

подпись, дата

И.О. Фамилия

инициалы и фамилия

Москва, 2026г

**Роли исполнителей проекта:**

**Елизавета Эдуардовна Афонина** - Тимлид + Frontend-разработчик  
**Кристина Олеговна Козина** - Backend-разработчик  
**Эвелина Игоревна Штейн** - Аналитик + Алгоритмист

## **Введение**

**Актуальность:** Кинотеатр — это объект с высокой динамикой. Нужно одновременно учитывать длительность фильмов, интервалы на уборку залов, технические перерывы и возрастные рейтинги. База данных позволяет избежать «накладок», когда на одно и то же время в одном зале претендуют два сеанса. Ручной учет проданных мест в эпоху онлайн-продаж невозможен. БД обеспечивает транзакционность: если место куплено через приложение, оно мгновенно блокируется в кассе, что исключает продажу двух билетов на одно кресло. Для владельцев бизнеса БД — это источник моментальных отчетов о выручке, заполняемости залов и эффективности проката конкретных кинокартин, что необходимо для принятия управленческих решений.

### **Техническое задание:**

**Цель системы** — автоматизация процессов планирования и реализации кинопроката, обеспечивающая централизованное управление расписанием сеансов, распределение зрительских мест в залах и мгновенную регистрацию продаж билетов для исключения ошибок дублирования и повышения операционной эффективности предприятия.

### **Основные объекты (сущности):**

#### **Кинозалы (Cinema\_Halls)**

Каждый зал является ограниченным ресурсом. Он характеризуется уникальным номером или названием, общей вместимостью и конфигурацией мест (ряды и номера кресел). Также залы могут различаться по техническому оснащению (наличие 3D-проекторов, систем Dolby Atmos, IMAX).

#### **Кинофильмы (Movies)**

Это информационный контент, который транслируется в залах. Описание фильма включает название, жанр, продолжительность (важно для планирования сетки), возрастной рейтинг (0+, 12+, 18+), страну производства и краткое описание. Продолжительность фильма напрямую влияет на расчет времени начала следующего сеанса.

## Сеансы (Sessions)

Центральный связующий элемент системы. Сеанс объединяет Фильм, Зал и конкретное Время (дата и час начала). При формировании сеанса учитывается стоимость билета (которая может зависеть от времени суток, дня недели или типа зала). Система должна контролировать отсутствие «накладок» — когда два фильма назначаются в один зал на одно и то же время.

## Билеты (Tickets)

Документ, подтверждающий право зрителя на посещение конкретного сеанса. Билет жестко привязан к Сеансу и определенному Месту в зале (ряд/место). Статус билета меняется в процессе жизненного цикла: «Свободен», «Забронирован», «Продан». Система обеспечивает уникальность продажи — одно место на один сеанс не может быть продано дважды.

## Основные процессы:

- Формирование репертуарного плана: Назначение фильмов на сеансы в доступные залы.
- Управление продажами: Регистрация факта покупки билета, расчет итоговой стоимости, печать или отправка электронного чека.
- Контроль заполняемости: Мониторинг оставшихся свободных мест в режиме реального времени.

## Проектирование базы данных:

### ER-Диаграмма:

На данном этапе мы определяем то, что из себя представляет наш проект. Для этого мы создаём ER-Диаграмму будущей Базы Данных.

В ней присутствуют 4 таблицы - “Movies“, “Halls“, “Tickets“ и “Sessions“.

В таблице “**Movies**“ присутствует 5 полей, из которых первое (id фильма), является первичным ключом., который *связан с таблицей “Sessions“* через поле “**movie\_id**”!

В таблице “**Halls**“ также первым полем выступает первичный ключ (id зала), который тоже *связан с таблицей “Sessions“* через поле “**hall\_id**”!

| MOVIES |            |    |
|--------|------------|----|
| int    | id         | PK |
| string | title      |    |
| string | genre      |    |
| int    | duration   |    |
| string | age_rating |    |

| HALLS  |             |    |
|--------|-------------|----|
| int    | id          | PK |
| string | name        |    |
| int    | total_rows  |    |
| int    | total_seats |    |
| string | category    |    |

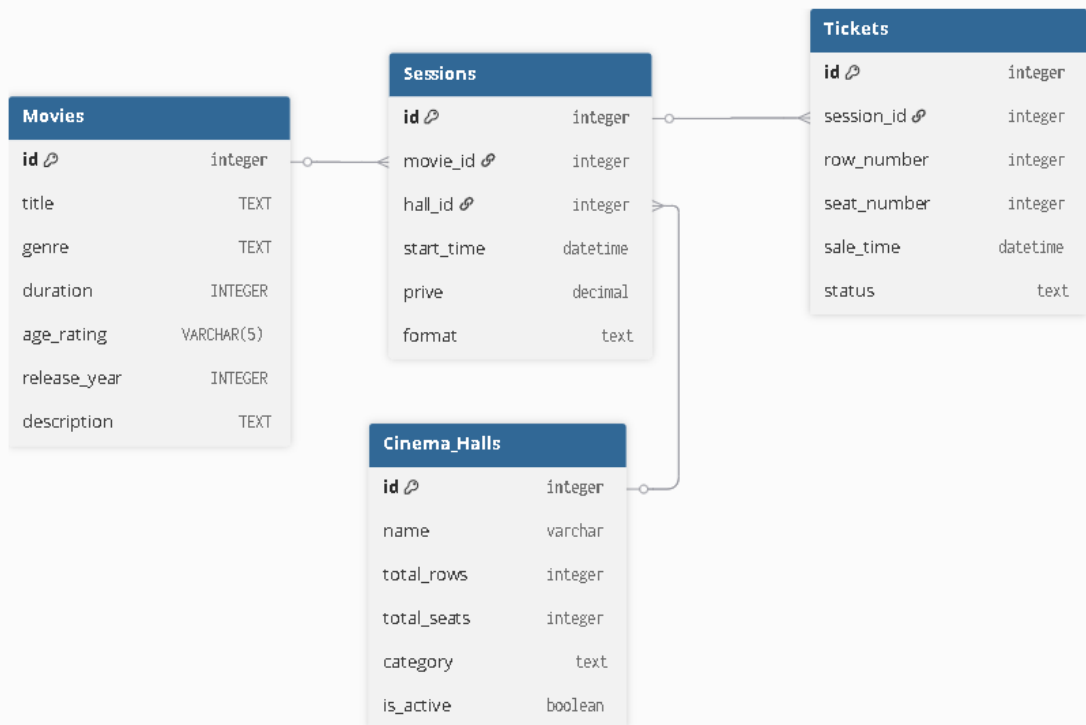


| SESSIONS |            |    |
|----------|------------|----|
| int      | id         | PK |
| int      | movie_id   | FK |
| int      | hall_id    | FK |
| datetime | start_time |    |
| decimal  | price      |    |



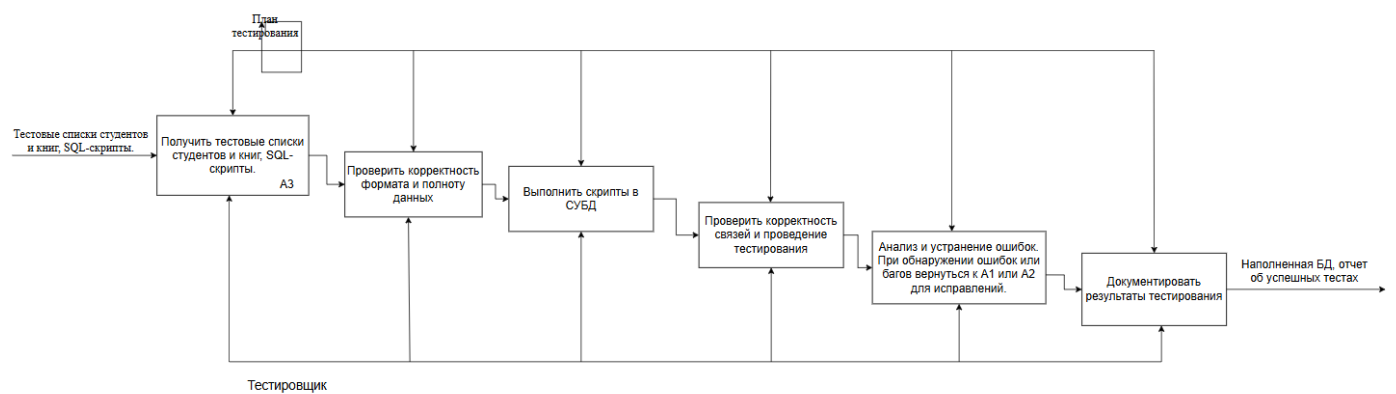
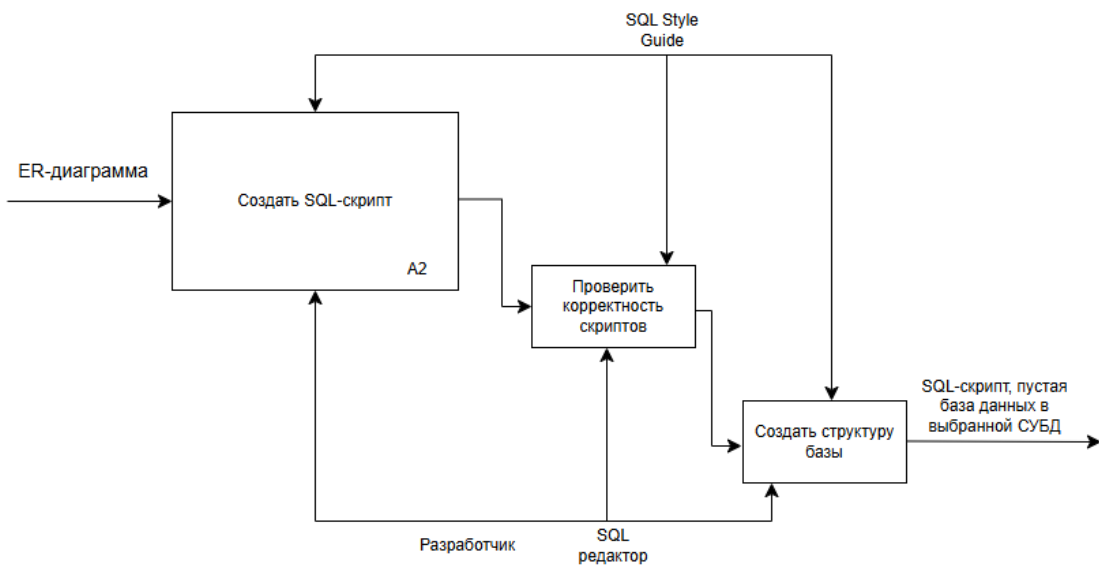
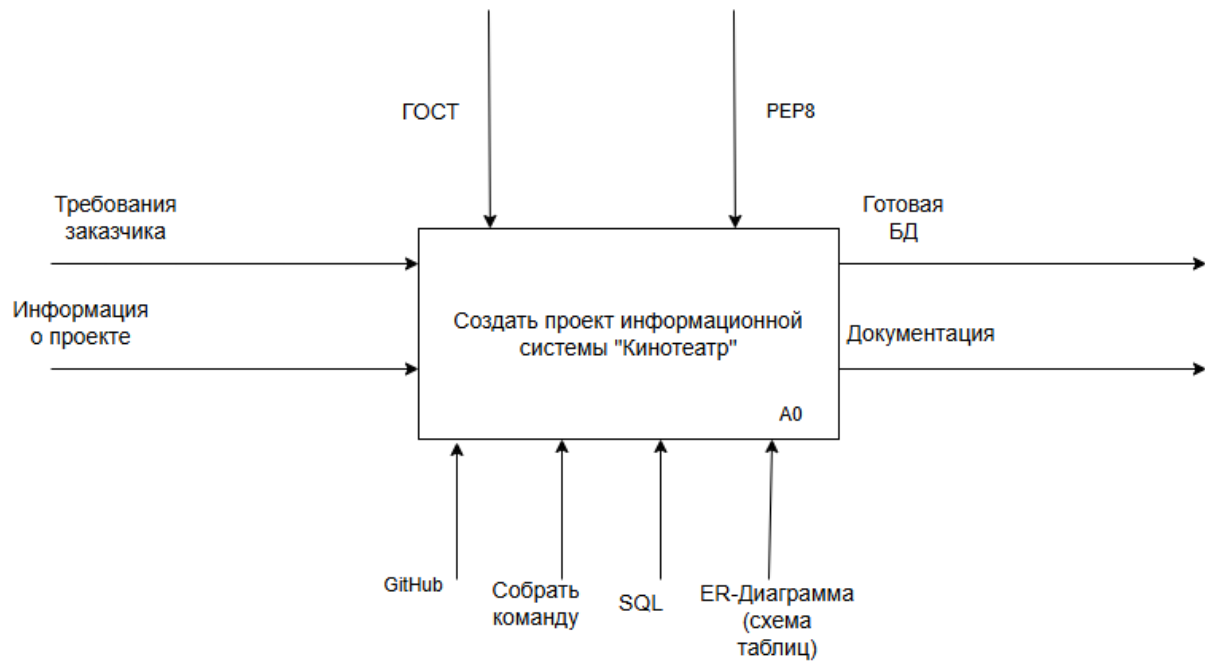
| TICKETS |             |    |
|---------|-------------|----|
| int     | id          | PK |
| int     | session_id  | FK |
| int     | row_number  |    |
| int     | seat_number |    |
| string  | status      |    |

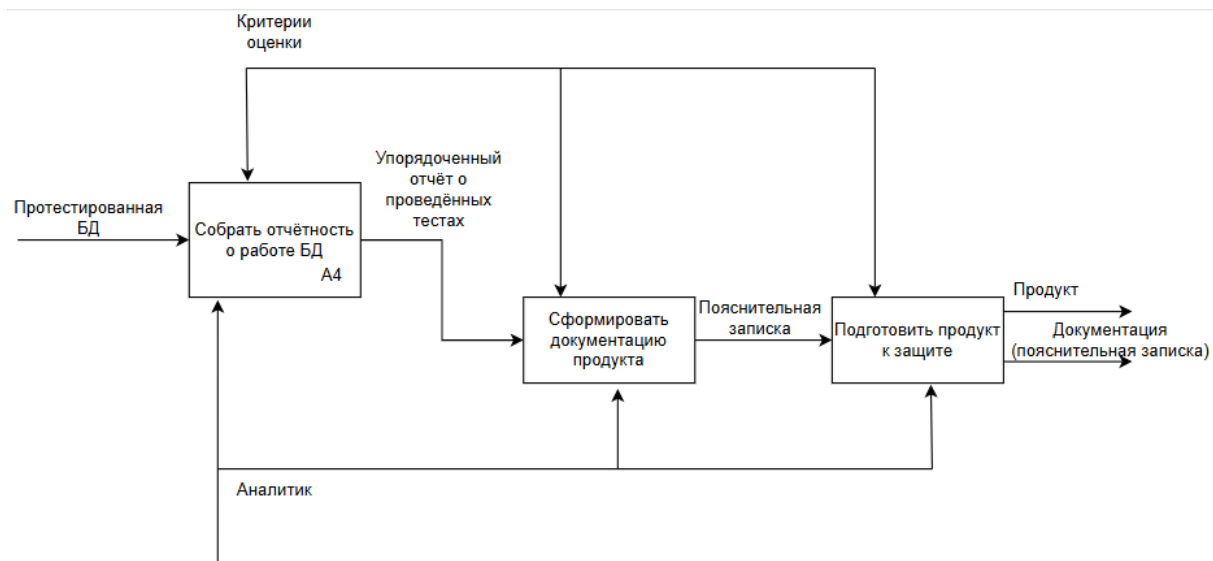
По итогу, в таблице **“Sessions”**, помимо своего первичного ключа (id), имеется два внешних ключа, отсылающих к информации из двух предыдущих таблиц. Первичный же ключ связан с таблицей **“Tickets”**, через поле **“session\_id”**. В таблице **“Tickets”**, которая будет выступать финальным звеном цепи нашего алгоритма, имеется свой первичный ключ (id), и внешний от **“Sessions”**! По итогу мы получаем вот такую схему:



Из таблицы **“Movies”** и **“Halls”**, таблица **“Sessions”** получает информацию о том, какой фильм, где и сколько будет показываться. После чего эти данные из **“Sessions”** переходят в **“Tickets”**, для печати после на чеке информации о сеансе, его времени, цене и зале.

## Диаграммы IDEF0:





## Теоретическая часть

### 1. История вопроса и эволюция систем управления данными



История автоматизации кинотеатров прошла путь от бумажных журналов учета и рулонных билетов до облачных экосистем.

- **Доцифровой этап:** До 1990-х годов учет посещаемости и расписание велись вручную. Основной проблемой была невозможность контроля дублирования мест в реальном времени и отсутствие оперативной отчетности.
- **Этап локальных БД (1990-е – 2000-е):** С появлением персональных компьютеров начали внедряться системы на базе ранних СУБД (например, FoxPro, Access). Появились понятия электронного расписания и цифровой печати билетов.
- **Современный этап:** Современные ИС кинотеатров — это высоконагруженные системы, интегрированные с онлайн-агрегаторами (Кинопоиск, Яндекс.Афиша) и программами лояльности. Сегодня фокус сместился с простого хранения данных на аналитику зрительских предпочтений и динамическое ценообразование.

## 2. Сравнительный анализ литературы и подходов к проектированию

В теории проектирования БД для кинотеатров выделяются два основных подхода, широко обсуждаемых в литературе по системному анализу (напр., К. Дейт, Р. Элмасри):

### А) Реляционный подход (Классический)

Большинство авторов (например, М. Грабер) настаивают на жесткой нормализации данных. В контексте проекта это выражается в разделении сущностей:

- Movies (фильмы) и Halls (залы) являются справочниками.
- Sessions (сеансы) выступает связующим звеном, реализующим связь «многие-ко-многим» между фильмом, временем и залом.
- Tickets (билеты) фиксируют транзакцию и привязываются к конкретному месту и сеансу.
- *Преимущество:* Целостность данных и отсутствие дублирования.

### Б) Объектно-ориентированный и NoSQL подходы

В современной практике (особенно в веб-интерфейсах) иногда допускается денормализация. Например, хранение списка забронированных мест прямо в документе сеанса (в формате JSON).

- *Критика:* Литература по транзакционным системам предостерегает от этого в финансовых модулях (продажа билетов), так как это усложняет соблюдение принципов ACID.

## 3. Оценка уровня разработанности проблемы

Проблема проектирования БД для кинотеатров в теории считается **высоко разработанной**, однако на практике возникают сложности в реализации бизнес-логики:

1. **Контроль коллизий:** В теоретических моделях часто опускается механизм «бронирования на время» (когда место удерживается 10 минут до оплаты).
2. **Схемы залов:** Описание таблицы Halls в литературе часто упрощено. На практике требуется хранение сложной топологии (ряды, места, VIP-зоны), что решается либо созданием отдельной таблицы Seats, либо хранением координат.

#### 4. Анализ предметной области и структурная декомпозиция

Проектирование ИС кинотеатра классифицируется в литературе как разработка **OLTP-системы** (Online Transaction Processing). Основная теоретическая сложность здесь заключается в обеспечении непротиворечивости данных при высокой интенсивности операций (выбор места — бронь — оплата).

- **Сущность Movies:** В теории информационного поиска рассматривается не просто как список названий, а как объект с метаданными (хронометраж, возрастной рейтинг, жанры). Хронометраж является критическим параметром для логики таблицы Sessions, так как он накладывает ограничения на построение расписания (интервалы между сеансами).
- **Сущность Halls:** Исследователи (например, Дж. Видерхольд) указывают на необходимость разделения статических характеристик зала (название, категория) и динамической топологии. В продвинутых моделях Halls связана с таблицей Seats (места), чтобы избежать жесткого кодирования количества кресел.
- **Сущность Sessions:** Является «центром кристаллизации» БД. В теории баз данных это классический пример **слабой сущности**, существование которой зависит от фильма и зала. Ошибка в проектировании этого узла ведет к накладкам (два фильма в одном зале в одно время).
- **Сущность Tickets:** Рассматривается в контексте финансового учета. Согласно теории транзакций, запись о билете должна обладать атомарностью: недопустима продажа билета без привязки к конкретному месту (связь с Seats) и сеансу.

#### 5. Сравнительный анализ моделей хранения данных

В практике разработки выделяют три уровня абстракции, которые необходимо описать в проекте:

| Критерий сравнения | Файловое хранение (CSV/JSON)    | Реляционная модель (SQLite/PostgreSQL) | NoSQL (MongoDB) |
|--------------------|---------------------------------|----------------------------------------|-----------------|
| Целостность        | Низкая (возможны дубли билетов) | Высокая (через Foreign Keys)           | Средняя         |
| Сложность отчетов  | Высокая (нужен перебор файлов)  | Низкая (через SQL JOIN)                | Средняя         |
| Масштабируемость   | Нулевая                         | Высокая                                | Очень высокая   |

Анализ показывает, что для ИС кинотеатра **реляционная модель** является эталонной, так как она минимизирует избыточность через нормальные формы (1NF, 2NF, 3NF). Например, вынесение названия фильма в Movies позволяет изменить его в одном месте, и оно автоматически обновится во всех сеансах.

**6. Проблема реализации бизнес-логики в теории программирования**  
Современная литература по программной инженерии (Р. Мартин, М. Фаулер) подчеркивает важность разделения ответственности:

1. **Слой данных (Persistence Layer):** SQLite хранит таблицы.
2. **Слой бизнес-логики (Domain Layer):** Python-код проверяет, не просрочено ли время сеанса перед продажей билета.
3. **Слой представления (Presentation Layer):** Оформление отчета по выручке.

Разрыв между теорией и практикой часто встречается в вопросе «умной» **схемы залов**. В теории достаточно иметь hall\_id, но на практике программисту приходится решать задачу визуализации: как превратить плоский список мест из БД в графическую сетку 10x12 для пользователя.

## 7. Математический аспект: Теория графов и расписаний

С точки зрения математического моделирования, база данных кинотеатра — это попытка решить задачу оптимального заполнения временных интервалов. Теория расписаний говорит нам, что сеансы не должны пересекаться в рамках одного hall\_id. Это накладывает требование к

реализации **триггеров** или программных проверок на уровне Python-кода перед выполнением команды INSERT в таблицу сеансов.

**Выполненная работа каждым членом команды:**

| Член команды                  | Ответственный за                                                                                                                                                           | Оценка выполненной работы |
|-------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------|
| Штейн Эвелина Игоревна        | <ul style="list-style-type: none"><li>• Написание и оформление отчётов</li><li>• Создание диаграмм</li></ul>                                                               | 5/5                       |
| Козина Кристина Олеговна      | <ul style="list-style-type: none"><li>• Проектирование БД</li><li>• Создание запросов</li><li>• Создание связей</li></ul>                                                  | 5/5                       |
| Афониная Елизавета Эдуардовна | <ul style="list-style-type: none"><li>• Проверка работы команды</li><li>• Создание GitHub</li><li>• Создание интерфейса</li><li>• Интеграция всех частей проекта</li></ul> | 5/5                       |

## Заключение

Для реализации учебного или MVP-проекта оптимальным является **реляционный подход с использованием SQLite**. Это обеспечивает необходимый уровень абстракции для работы с сущностями Movies, Sessions, Halls и Tickets, позволяя реализовать строгую логику продажи билетов и генерацию отчетов о заполняемости залов. Таким образом, создание БД для кинотеатра — это не просто создание четырех таблиц, а реализация строгой математической модели, где каждая покупка билета должна мгновенно изменять состояние системы, блокируя доступ к ресурсу (месту) для других транзакций.

## Список литературы

### 1. Базы данных и SQL (Теория и практика)

- Крис Дейт — «Введение в системы баз данных» (Introduction to Database Systems).
- Алан Бьюли — «Изучаем SQL».
- Томас Коннолли, Каролин Бегг — «Базы данных. Проектирование, реализация и сопровождение».

### 2. Язык Python и работа с данными

- Марк Лутц — «Изучаем Python» (в 2-х томах).
- Лучано Рамальо — «Python. К вершинам мастерства» (Fluent Python).
- Дэвид Бизли, Брайан Джонс — «Python. Книга рецептов».

### 3. Архитектура и командная разработка

- Роберт Мартин — «Чистый код» (Clean Code).
- Мартин Фаулер — «Шаблоны корпоративных приложений».
- Эрик Фримен, Элизабет Робсон — «Паттерны проектирования».

## Приложения

